THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

VeriScreen: A Web-Based System for Academic Document Verification and HELSB Loan Screening

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

the Bachelor's degree in the Department of Computing and Informatics

Students:

Samuel Chibinji Mwanza Computer Number 2021465934
Nosiku Mukuka Computer Number 2022030346

Supervisor: Mrs. Monde Monica Kalumbilo Kabemba

Date of Submission: 23rd October 2026

DECLARATION

We declare that this final year project report is our own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not our own have been appropriately acknowledged and referenced.

This project was undertaken as a group. The report describes the system produced by the group. Where a component was completed jointly, it is described as joint work. We have not presented unpublished work of any other person as our own.

Student Name: Samuel Chibinji Mwanza

Computer Number: 2021465934

Signature: _____

Date: _____

Student Name: Nosiku Mukuka

Computer Number: 2022030346

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: Mrs. Monde Monica Kalumbilo Kabemba

Signature: _____

Date: _____

Departmental/Examiner Approval (if applicable): _____

ABSTRACT

Students who apply for support from the Higher Education Loans and Scholarships Board must present academic documents that a lender can trust, and the lender must be able to explain why an application is approved or refused. Paper statements and informal screenshots are difficult to check, and a status such as "approved" does not tell the student which university record produced the decision. This project designed and implemented VeriScreen, a student web portal, together with a separate HELSB officer portal. The student portal shows registration, fees, examinations, and results taken from a simulated university record, and it issues those records as PDF documents sealed with a QR verification code. A phone that scans the code opens a page stating whether the document was issued. When a student applies for a loan, an officer moves the application through submission, portal confirmation, payment, screening, contract, and disbursement. At screening, the system reads the latest fee balance, registration, examination standing, and published grades, and it writes a plain-language result such as "Not eligible: outstanding fee balance of K500.00" or "Approved: good academic standing, fees cleared, registration confirmed." The student dashboard shows the same stage and sentence. The completed system runs locally as two React applications and two Node.js services over MySQL. Testing with four seeded students confirmed login, PDF content, QR verification, live tracker updates, and the screening sentences.

Keywords: academic document verification; QR codes; student information systems; HELSB; loan screening

ACKNOWLEDGEMENTS

We thank the Department of Computing and Informatics for the opportunity to undertake this final year project, and our supervisor for guidance during the work. We also acknowledge the Higher Education Loans and Scholarships Board as the institutional setting that motivated a clearer link between university records and loan decisions. The academic figures used in the demonstration come from a simulated university record prepared for this project. They are not a copy of a live University of Zambia production database.

Contents

Declaration **1**

Approval **2**

Abstract **3**

Acknowledgements **4**

List of Abbreviations and Acronyms **11**

Note on Group Projects **12**

1 Introduction **13**

1.1 Background and Context 13
1.2 Problem Statement 13
1.3 Project Aim 14
1.4 Project Objectives 14
1.5 Research Questions (where applicable) 14
1.6 Scope of the Project 14
1.6.1 In Scope 14
1.6.2 Out of Scope 15
1.7 Significance of the Project 15
1.8 Limitations 15
1.9 Organisation of the Report 15

2 Literature Review **16**

2.1 Introduction 16
2.2 Key Concepts and Theories 16
2.3 Existing Systems / Related Work 16
2.4 Comparative Analysis 17
2.5 Research / Knowledge Gap 17
2.6 Conceptual or Theoretical Framework (where applicable) 18
2.7 Chapter Summary 18

3 Methodology **19**
3.1 Introduction . . . 19
3.2 Research / Development Approach . . . 19
3.3 Requirements / Data Collection . . . 19
3.4 System Architecture / Research Workflow . . . 20
3.5 Tools and Technologies . . . 20
3.6 Ethical and Legal Considerations . . . 21
3.7 Evaluation Method . . . 21
3.8 Chapter Summary . . . 21

4 System Analysis and Design **22**
4.1 Introduction . . . 22
4.2 Stakeholder and User Analysis . . . 22
4.3 Functional Requirements . . . 22
4.4 Non-Functional Requirements . . . 23
4.5 Use Cases / User Stories . . . 23
4.6 System / Solution Architecture . . . 24
4.7 Database / Data Design (where applicable) . . . 24
4.8 Interface / Interaction Design (where applicable) . . . 25
4.9 Design illustrations . . . 25
4.10 Algorithms / Models / Technical Design . . . 33
4.11 Chapter Summary . . . 34

5 Implementation / Development **35**
5.1 Introduction . . . 35
5.2 Development Environment . . . 35
5.3 System Components / Modules . . . 35
5.4 Key Implementation Details . . . 35
5.5 Integration . . . 36
5.6 Challenges and Solutions . . . 36
5.7 Individual Contribution (group projects) . . . 37
5.8 Chapter Summary . . . 37

6 Testing, Results and Evaluation **38**
6.1 Introduction . . . 38
6.2 Testing Strategy . . . 38
6.3 Test Cases and Results . . . 39
6.4 Experimental Results / Performance Results (where applicable) . . . 40
6.5 User Evaluation (where applicable) . . . 40
6.6 Analysis of Results . . . 40

6.7 Objective-by-Objective Evaluation 40
6.8 Individual Results (group projects) 41
6.9 Chapter Summary 41

7 Discussion **42**
7.1 Introduction 42
7.2 Interpretation of Findings 42
7.3 Comparison with Existing Work 42
7.4 Contributions of the Project 42
7.5 Limitations and Implications 43
7.6 Chapter Summary 43

8 Conclusion and Recommendations **44**
8.1 Introduction 44
8.2 Summary of the Project 44
8.3 Conclusions 44
8.4 Achievement of Objectives 44
8.5 Recommendations 45
8.6 Future Work 45

References **46**

A Individual Contribution Statement (Group Projects) **47**
A.1 Project and Group Information 47
A.2 Overall Group Project 47
A.3 Individual Role and Responsibilities 48
A.4 Detailed Individual Contribution 48
A.5 Individual Deliverables 49
A.6 Contribution to the Main System or Research Output 49
A.7 Individual Implementation and Technical Work 49
A.8 Individual Testing and Evaluation Contribution 49
A.9 Collaboration with Other Group Members 49
A.10 Contribution Summary 50
A.11 Estimated Contribution 50
A.12 Statement of Authorship and Contribution 50
A.13 Supervisor Verification 52

B User Manual / Installation Guide **53**

C Additional System Designs / Diagrams **55**

D Additional Test Cases and Results **56**

E Data Collection Instruments **57**

F Additional Code / Configuration **58**

G Other Supporting Material **59**

List of Figures

4.1 Separation of the student portal and the HELSB officer portal 25
4.2 Principal use cases 25
4.3 HELSB pipeline. A result that begins “Not eligible” stops at screening. . . 26
4.4 Screening checks, applied from top to bottom. The first match is the sentence stored for the student and the officer. 26
4.5 VeriScreen sign-in at desktop width 26
4.6 Student dashboard for Samuel Chibinji Mwanza, with the tracker at screening 27
4.7 Issued documents on the student portal 27
4.8 Published results for the signed-in student 28
4.9 Settings, with dark mode and email notifications. Help is the item above Settings in the sidebar. 28
4.10 VeriScreen at phone width: the dashboard, and the menu that replaces the sidebar 29
4.11 Page opened from a document QR code, including the issued time 30
4.12 HELSB officer sign-in 30
4.13 HELSB portal after sign-in: the university list 31
4.14 The same university list at phone width 32
4.15 Officer view of Samuel Chibinji Mwanza at screening, with the same sentence shown on the student dashboard 33

List of Tables

2.1 Comparison of ways to present an academic document 17

3.1 Running parts of the completed system 20
3.2 Tools and technologies 20

4.1 Stakeholders and what they need 22
4.2 Screening rules, applied in order 24
4.3 HELSB pipeline 33

6.1 Representative system tests 39

B.1 Accounts for the demonstration 54

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	**Meaning**
API	Application Programming Interface
CRUD	Create, read, update, and delete
GPA	Grade point average
HELSB	Higher Education Loans and Scholarships Board
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
NRC	National Registration Card
PDF	Portable Document Format
QR	Quick Response code
REST	Representational State Transfer
SIS	Student Information System
SQL	Structured Query Language
UI	User Interface
UNZA	University of Zambia

NOTE ON GROUP PROJECTS

This report is the group account of one shared system. Samuel Chibinji Mwanza (2021465934) and Nosiku Mukuka (2022030346) worked on VeriScreen and the HELSB officer portal as a single product. The student portal, the officer portal, the document service, and the screening rules are parts of that product. Appendix A records the joint contribution and leaves a precise percentage split to be confirmed with the supervisor before the report is bound. Neither student claims a module that the repository presents as separate personal property of the other.

Chapter 1

Introduction

1.1 Background and Context

Zambian undergraduate students routinely need official evidence of registration, fees, and results. The same evidence is required when a student seeks a loan or scholarship from the Higher Education Loans and Scholarships Board. In current practice, a student may download or photograph a portal page and send that image to a sponsor. The recipient cannot easily tell whether the image still matches the university record, whether a balance has changed, or which rule produced a later loan decision.

VeriScreen addresses that gap as a student-facing web system. It presents the academic record, issues it as a PDF, and attaches a verification code that a third party can scan. Beside it, a separate officer portal lets HELSB staff move an application through a visible pipeline and record an explainable screening result. The two sites do not share a login. A student cannot open the officer screens, and an officer cannot sign in to the student portal with an officer account.

The demonstration uses a simulated University of Zambia record for a small set of students. The figures on each document are read from that record. They are not typed into the PDF by hand and they are not copied from a screenshot.

1.2 Problem Statement

Students lack a single place in which an academic document can be issued from the university record and later checked by someone who was not present when it was downloaded. HELSB officers, in turn, can be shown a loan percentage or a status word without a sentence that ties the decision to fees, registration, examinations, and grades. The project therefore addresses two related problems: proof that a document was issued, and an explanation of why screening reached its result.

1.3 Project Aim

The aim of this project is to build a working web system that issues verifiable academic documents from a simulated university record and explains HELSB screening decisions in plain language.

1.4 Project Objectives

1. Provide a student portal on which a student can sign in and view registration, fees, examinations, and results held for that student.
2. Generate PDF documents whose content is taken from those records, including a fee statement, academic results, and a confirmation of registration.
3. Allow a phone to scan a QR code on an issued document and display whether VeriScreen issued that document.
4. Provide a separate HELSB officer portal that can receive an application and move it through a defined pipeline visible to the student.
5. When screening runs, compute a plain-language result from the fee balance, registration, examination standing, and academic results, and show that sentence to both the officer and the student.

1.5 Research Questions (where applicable)

The project is a design-and-build project rather than a statistical study. The questions that guided the build were the following.

1. Can an issued PDF be checked later without trusting a screenshot?
2. Can a loan decision be stated as a sentence that a student can match to a university record?
3. Can the student view and the officer view stay consistent while the application moves?

1.6 Scope of the Project

1.6.1 In Scope

The completed project includes the student web portal, the HELSB officer web portal, the student API, the officer API, MySQL storage for both, PDF generation, QR verification,

the application tracker, and the screening explanation. The student portal is laid out for a desktop window and for a phone-width window. An Expo client that calls the same student API is included in the repository. Seeded demonstration students and officers are included so that the workflow can be exercised without a live university or HELSB connection.

1.6.2 Out of Scope

The project does not replace the University of Zambia student information system and does not connect to a production HELSB database. It does not disburse real money, sign a legally binding loan contract, or perform biometric identity checks. The illustrations in this report are screenshots of the web portals, including the student portal at phone width, together with diagrams of the design. Forgotten-password and self-registration flows were removed from the student sign-in page so that access stays limited to issued accounts.

1.7 Significance of the Project

Students gain a document that can be shown to a sponsor and checked afterwards. Officers gain a screening result they can read aloud and defend from the record. The department gains a concrete example of a multi-application information system in which authority is split: the university record supplies facts, VeriScreen issues documents, and HELSB staff control the loan pipeline.

1.8 Limitations

The academic data are simulated. Screening uses rules chosen for this project, including a latest-balance test and a grade-point threshold, and those rules are not a published HELSB regulation. QR verification confirms that VeriScreen issued the code. It does not detect a visually edited photocopy if the code itself is left unchanged, and it does not re-check the PDF bytes. Phone scanning requires the phone and the computer to be on a reachable network, and a host firewall can block the verification port. The supervisor's name is not printed on the title page because it was not available when this report was generated.

1.9 Organisation of the Report

Chapter 2 reviews document verification, loan screening, and the related technical ideas. Chapter 3 describes how the system was built and how it was evaluated. Chapter 4 records the requirements and the design. Chapter 5 describes the implementation. Chapter 6 reports the tests. Chapter 7 discusses what the results mean. Chapter 8 concludes and recommends further work. The appendices contain the group contribution statement and operating notes.

Chapter 2

Literature Review

2.1 Introduction

This chapter sets out the ideas required to understand the system: official academic records, the role of a loans board, machine-checkable documents, and the separation between a student application and an officer decision.

2.2 Key Concepts and Theories

An academic record is authoritative only while it remains tied to the institution that holds it. A downloaded file becomes a copy. A useful verification scheme therefore stores an identifier at issue time and resolves that identifier later against the issuing system. A QR code is a two-dimensional barcode that can carry a short web address [2]. The phone does not need a special application if the address opens in a browser.

Loan screening is a decision procedure. A percentage or a colour can summarise the outcome, but it does not state the premise. An explainable decision names the fact that failed or the facts that passed. In this project those facts are the outstanding fee balance, confirmation of registration, examination eligibility, and academic standing derived from published grades.

The architecture follows a REST style: clients call named HTTP resources, and each service keeps its own data [4]. Authentication uses a JSON Web Token [3]. The token shows that a sign-in succeeded. It does not by itself prove that a document is genuine. Document trust is handled by the verification identifier, not by the student's session.

2.3 Existing Systems / Related Work

University portals commonly let a student view fees, registration, and results. Those pages are useful to the student and weak as evidence for a third party, because a screenshot can be cropped or edited. A separate loans board then receives whatever the student chooses to send.

The Higher Education Loans and Scholarships Board is the Zambian body charged with administering higher-education loans and scholarships [1]. Its officers need student identity, programme, registration, fee clearance, and academic progress. Those facts originate at the university. A system that only stores a loan percentage breaks that link.

General document systems such as digitally signed PDFs can prove that bytes were not changed after signing. That approach needs certificate management. This project uses a lighter check that is enough for a demonstration: the QR code points at the issuing server, and the server reports whether that code was issued, to whom, and as which document type.

2.4 Comparative Analysis

Table 2.1 compares three ways a sponsor might receive an academic document.

Table 2.1: Comparison of ways to present an academic document

Criterion	Screenshot	Signed PDF only	VeriScreen QR
Readable by a person	Yes	Yes	Yes
Tied to a student record	No	Only if generated from it	Yes, at issue time
Checkable on a phone	No	Only with extra tools	Yes, by scanning
Explains a later loan decision	No	No	Yes, in the officer and student views

A signed PDF is stronger against byte-level tampering. VeriScreen is stronger for the demonstration constraint that a phone camera and a browser must be enough, and that the loan decision must be readable as a sentence.

2.5 Research / Knowledge Gap

Existing student portals and loan files do not, in the setting studied here, give a third party a simple check of an issued document and give the student a sentence that cites the fee balance or the academic standing. The gap is not the absence of databases. It is the absence of a checkable document and an explainable screening step between the university record and the loans board.

2.6 Conceptual or Theoretical Framework (where applicable)

The project uses a three-role framework.

1. The university record is the source of academic facts.
2. VeriScreen is the issuer of documents and the student's view of the loan pipeline.
3. The HELSB officer portal is the place where staff advance that pipeline and where screening is run.

A fact may be copied into VeriScreen so that a document can be generated when the original system is not online. The screening sentence is computed from the copy held for the student at the moment screening runs.

2.7 Chapter Summary

Screenshots are poor evidence. A QR address checked against the issuer is a practical alternative for a phone. Loan screening is more useful when it states the record that produced the result. The project applies those ideas by separating the student portal from the officer portal and by generating both the document and the screening sentence from the same student record.

Chapter 3

Methodology

3.1 Introduction

The project was conducted as a design-and-build exercise. The method was to fix the requirements from the student and officer workflows, implement them as running web applications, and evaluate them against those workflows rather than against a survey sample.

3.2 Research / Development Approach

Development followed short increments. Each increment had a visible result: student sign-in, records, PDF layout, phone verification, the officer tracker, and then the screening sentence. A change was kept when it could be demonstrated in the browser or by calling the API. This is closer to an incremental delivery approach than to a single waterfall handover, although the requirements were written down before each increment so that the scope did not drift into unrelated features.

An earlier design in the repository used Java services and PostgreSQL. The completed local system was moved to Node.js and MySQL so that it could run with a widely available database and so that the student site and the officer site could be started without a Java runtime. The behaviour required by the objectives was preserved: issued documents, separated officer access, and a status the student can see.

3.3 Requirements / Data Collection

Requirements were taken from the workflows a student and an officer must complete, not from a field questionnaire. The academic figures were seeded from a simulated university record for four primary students: Samuel Chibinji Mwanza, Nosiku Mukuka, Grace Banda, and Peter Zulu. Fee amounts, registration flags, examination status, and grades are stored in MySQL and read at document generation and at screening. No live student was interviewed, and no production university extract was used.

3.4 System Architecture / Research Workflow

The running system has four processes.

Table 3.1: Running parts of the completed system

Part	Address	Responsibility
Student web portal	localhost:5173	Sign-in, records, documents, tracker
Student API	port 8081	Identity, PDFs, verification, student copy of the pipeline
HELSB web portal	localhost:5174	Universities, schools, applications, screening
HELSB API	port 8083	Officer identity, pipeline, call to screening

The student API stores students, fees, registrations, examinations, results, issued documents, and the student's copy of the HELSB application in `veriscreen_db`. The officer API stores officers and applications in `helsb_db`. When an officer advances an application, the officer API notifies the student API with a shared internal key. When the next stage is screening, the officer API asks the student API to explain the record before the stage is saved.

3.5 Tools and Technologies

Table 3.2 lists the tools used in the completed build.

Table 3.2: Tools and technologies

Tool	Use in the project
React and TypeScript	Student portal and officer portal
Vite	Development server and proxy to each API
Node.js and Express	Both HTTP APIs
MySQL	`veriscreen_db` and `helsb_db`
PDFKit	Fee, results, confirmation, and related PDFs
QR code library	Address embedded in each PDF
JSON Web Tokens	Student and officer sessions
bcrypt	Password hashes

PostgreSQL remains supported when a deployment URL is supplied, which allows

the same API code to run against a hosted PostgreSQL database. The local demonstration uses MySQL.

3.6 Ethical and Legal Considerations

The students in the database are demonstration identities created for the project. National registration numbers in that seed are fictional. The system does not collect live biometric data and does not send documents to an external loans board. Officer and student passwords in the seed are development passwords and are not suitable for a public deployment. The verification page reveals the student name, computer number, and document type for a code that was issued. It does not reveal grades or fee amounts. That is intentional: a person who scanned a document the student chose to share can confirm the issue, but the code alone is not a full academic transcript.

3.7 Evaluation Method

Success was defined as the observable behaviour in the objectives. A test passed when the screen or the API response matched the record. Representative cases used the seeded students because their fee balances and grades are known in advance. Samuel has a latest outstanding balance of K500.00. Nosiku has cleared fees, confirmed registration, and published grades. Grace Banda and Peter Zulu have outstanding balances. No claim is made about usability scores, because a formal user study was not conducted.

3.8 Chapter Summary

The system was built in increments and judged by whether a student can obtain a document, a phone can check it, an officer can move the pipeline, and screening can explain itself from the stored record.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter records who uses the system, what it must do, and how the main parts are arranged.

4.2 Stakeholder and User Analysis

Table 4.1: Stakeholders and what they need

Stakeholder	Need
Student	See the academic record, download a PDF, and follow the loan pipeline
Sponsor or verifier	Scan a code and see whether the document was issued
HELSB officer	Open an application, advance it, and read the screening sentence
University (simulated)	Remain the origin of fees, registration, examinations, and grades
Department	A demonstrable final-year system with a written account

4.3 Functional Requirements

1. The student signs in with a computer number and password.
2. The student sees a dashboard with a personal welcome, the HELSB tracker, recent documents, and recent activity.
3. The student can open registration, fee history, and results drawn from the stored record.

4. The student can preview and download an issued PDF.
5. A verification address on the PDF returns a page that states whether the code was issued and names the student and document type.
6. The student can submit an application to HELSB. Seeded applications already exist for the demonstration students.
7. An officer signs in with an officer email and password on a separate site.
8. The officer browses universities, schools, and applications.
9. The officer advances the pipeline one stage at a time.
10. Entering screening computes and stores a plain-language explanation.
11. A result that begins “Not eligible” stops the pipeline at screening.
12. An approved result allows the officer to continue to contract and disbursement.
13. The student tracker updates to the same stage and shows the same sentence.

4.4 Non-Functional Requirements

The student interface keeps the established VeriScreen colours: a dark green sidebar and gold accents. The officer interface is a separate layout so that the two organisations are not confused. Passwords are stored as hashes. Officer and student tokens are different secrets. The internal call from the officer API to the student API requires a shared key. Pages that a phone will open must be readable on a narrow screen. The local system must start without a Java runtime.

4.5 Use Cases / User Stories

The main stories are the following.

1. As a student, I sign in so that I see only my record.
2. As a student, I download a fee statement so that the figures match my stored payments and balance.
3. As a verifier, I scan the QR code so that I see a clear verified or not-verified page.
4. As an officer, I open a school and an application so that I can move the loan pipeline.
5. As a student, I watch the dashboard so that I see the stage the officer has reached and, after screening, the reason.

4.6 System / Solution Architecture

The browser never reads MySQL directly. The student portal calls the student API. The officer portal calls the officer API. Those APIs do not share tables. The only bridge is the internal HTTP call used for the tracker and for screening. This keeps an officer action from becoming a direct write into student tables by the officer process, and it keeps the sentence in one place: the student API computes it, and both databases store the text that was returned.

Screening applies the first matching rule in Table 4.2.

Table 4.2: Screening rules, applied in order

Condition on the student record	Sentence
No fee row	Not eligible: no fee record is available.
Latest outstanding balance greater than zero	Not eligible: outstanding fee balance of K*amount*
Latest registration not confirmed	Not eligible: registration is not confirmed.
Latest examination blocked or not eligible	Not eligible: examinations are blocked.
No published results, a failing grade, or GPA below 2.00	Not eligible: academic standing is below the required level.
Otherwise	Approved: good academic standing, fees cleared, registration confirmed.

GPA uses the credit-weighted scale A and A+ = 4.0, A− = 3.7, B+ = 3.3, B = 3.0, and so on down to F = 0.

4.7 Database / Data Design (where applicable)

`veriscreen_db` holds students, registrations, fees, examinations, academic results, issued documents, notifications, and HELSB application rows including the pipeline stage and the screening explanation. `helsb_db` holds officers, applications, and status history. An application row stores the student computer number, programme, school, document identifiers, pipeline stage, and the explanation text. The two databases can therefore disagree only if the internal notification fails. The officer action still saves the officer-side stage, and the error is written to the server log.

4.8 Interface / Interaction Design (where applicable)

The student home is a dashboard, not a dump of every table. Fees and results have their own pages. Documents open as PDFs in a preview and as downloads. The HELSB area shows the application and the screening sentence when one exists. The officer home is a list of universities, then schools, then applications. The application page shows the six pipeline steps and one button for the next step. After screening, the sentence is shown on that page in green when it starts with “Approved” and in red when it starts with “Not eligible”. Settings offers a dark-mode switch and email notifications. Help sits above Settings, and sign-out asks for confirmation. On a narrow screen the sidebar becomes a menu. The QR code on each PDF carries the time the document was issued, and the verification page prints that time.

4.9 Design illustrations

The diagrams in this section describe the structure. The screenshots that follow were taken from the running system on 19 October 2026, signed in as Samuel Chibinji Mwanza on the student portal and as officer Mwaka Chanda on the HELSB portal.

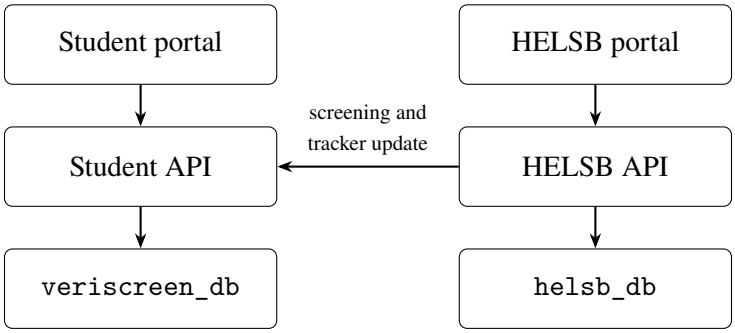


Figure 4.1: Separation of the student portal and the HELSB officer portal

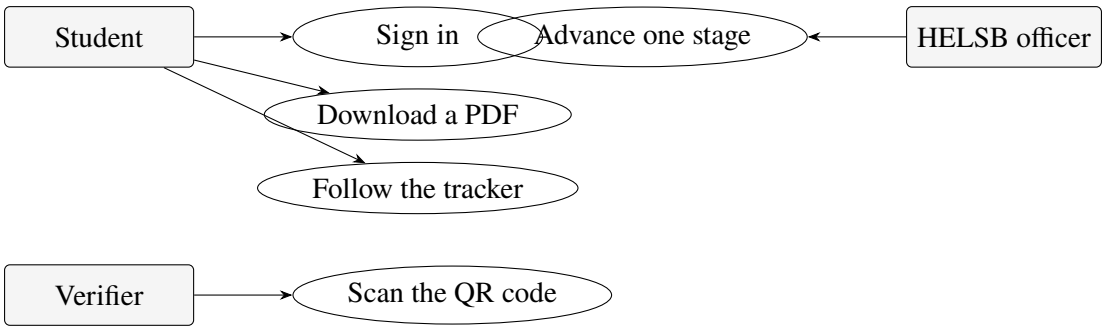


Figure 4.2: Principal use cases

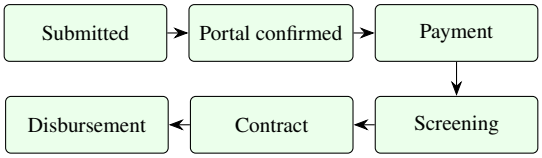


Figure 4.3: HELSB pipeline. A result that begins “Not eligible” stops at screening.

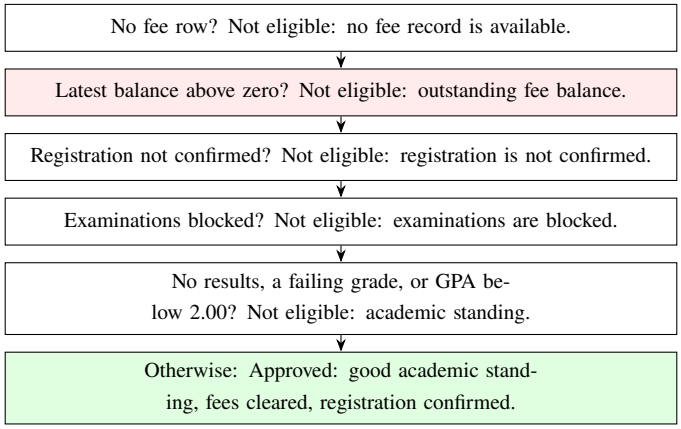


Figure 4.4: Screening checks, applied from top to bottom. The first match is the sentence stored for the student and the officer.

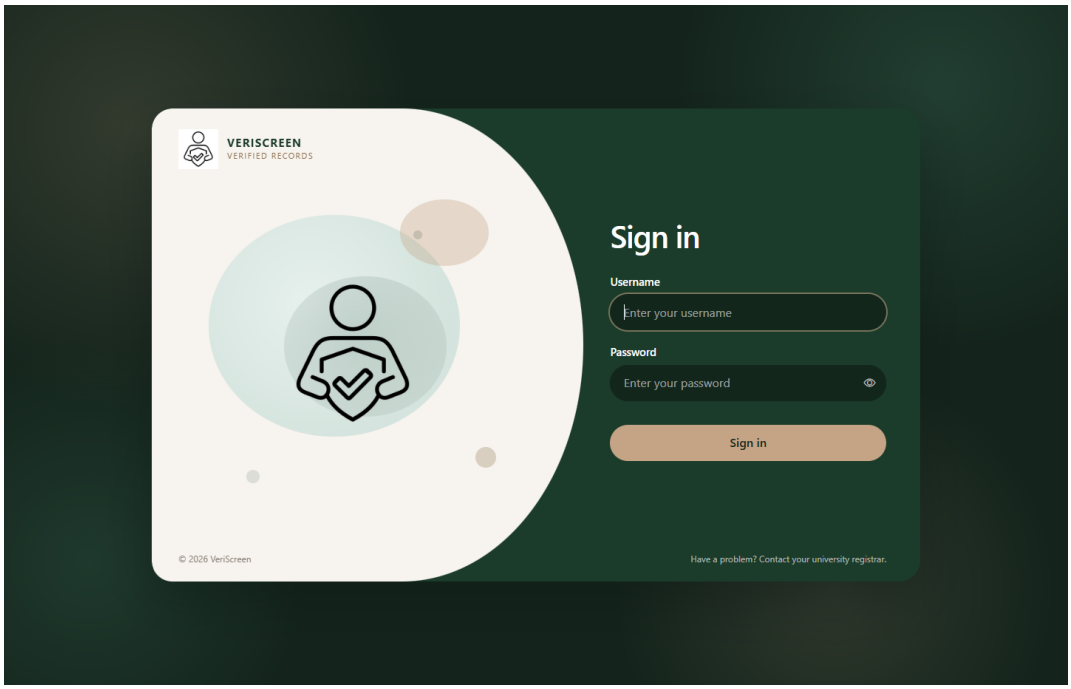


Figure 4.5: VeriScreen sign-in at desktop width

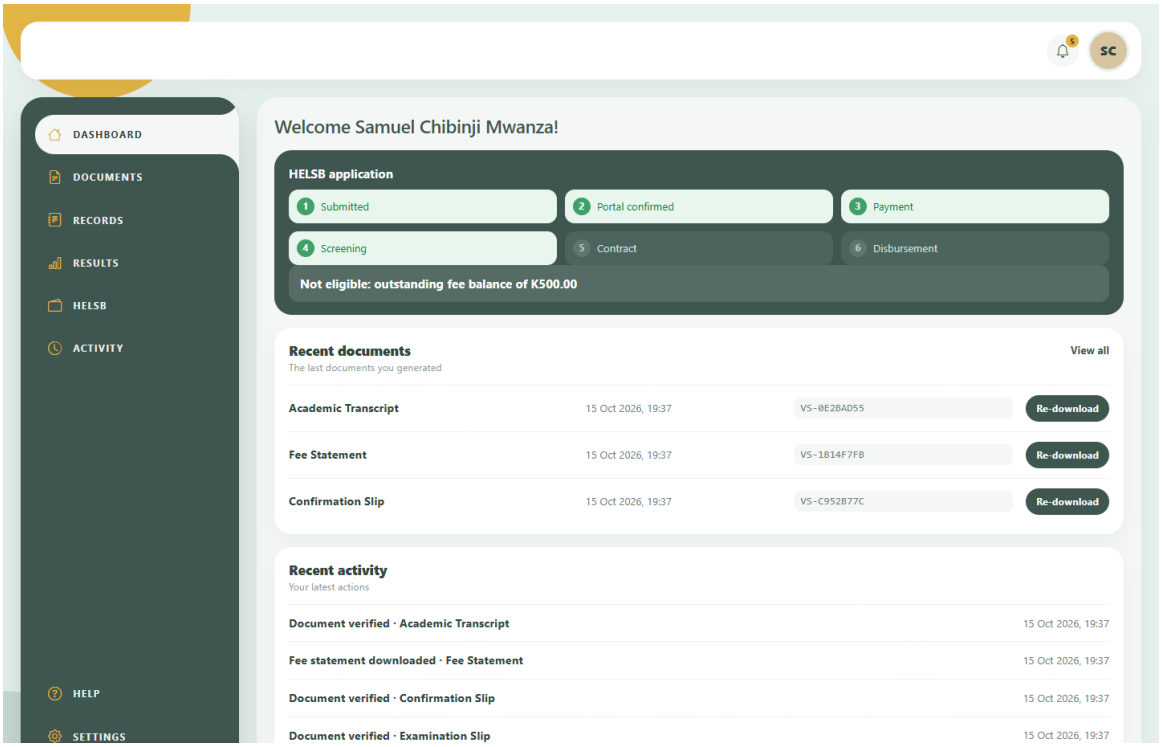


Figure 4.6: Student dashboard for Samuel Chibinji Mwanza, with the tracker at screening

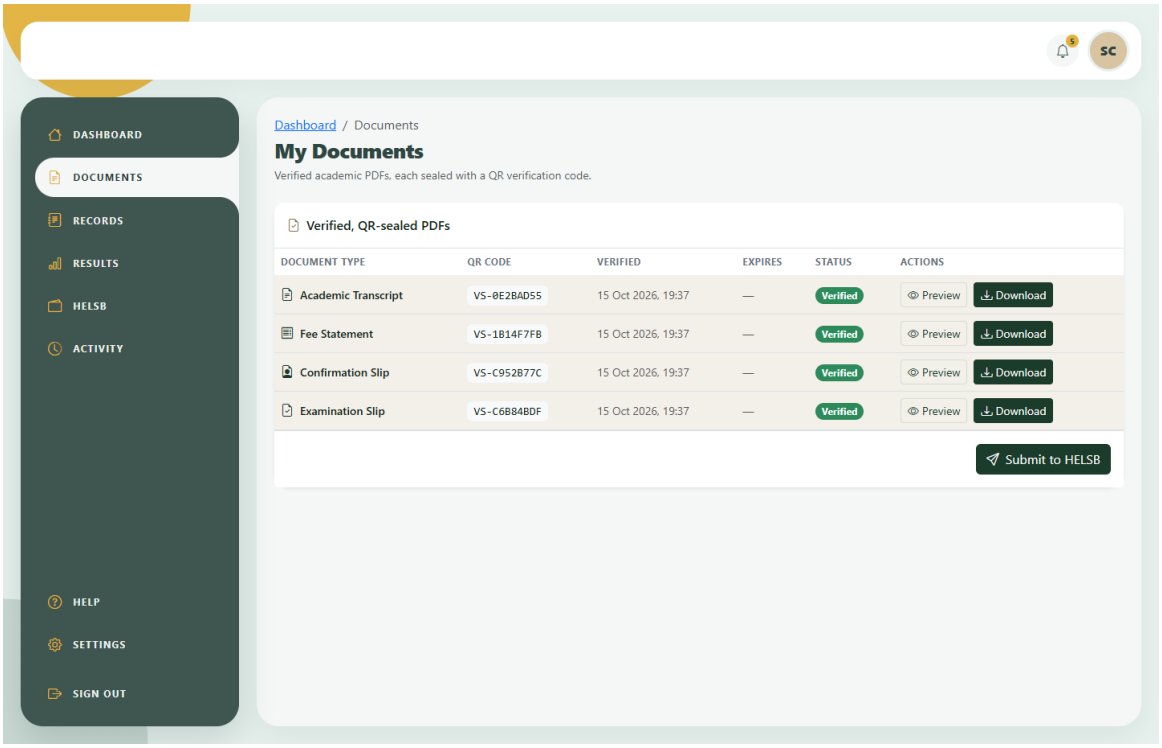


Figure 4.7: Issued documents on the student portal

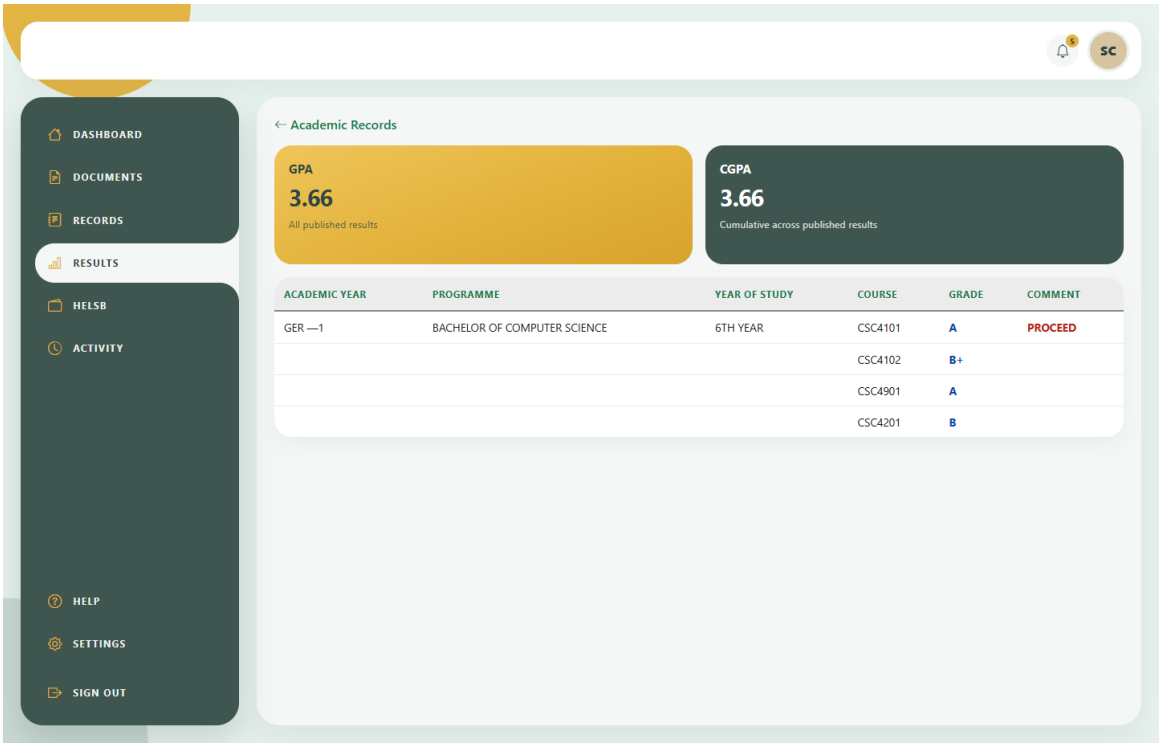


Figure 4.8: Published results for the signed-in student

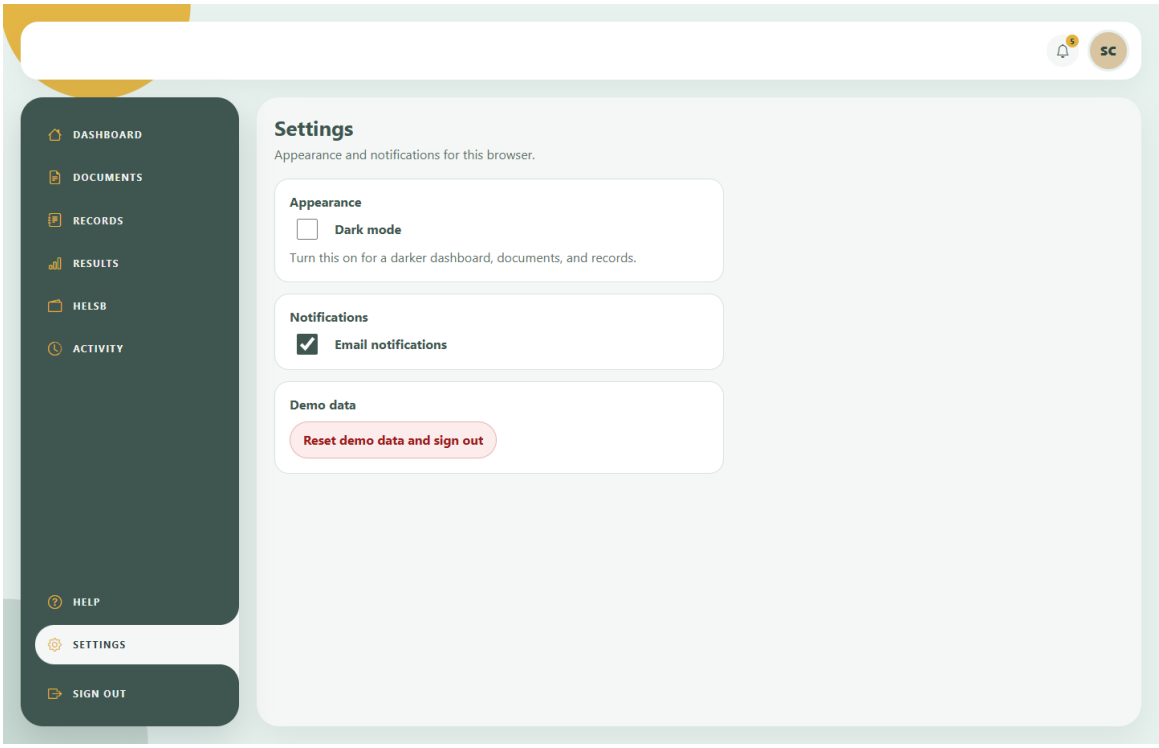


Figure 4.9: Settings, with dark mode and email notifications. Help is the item above Settings in the sidebar.

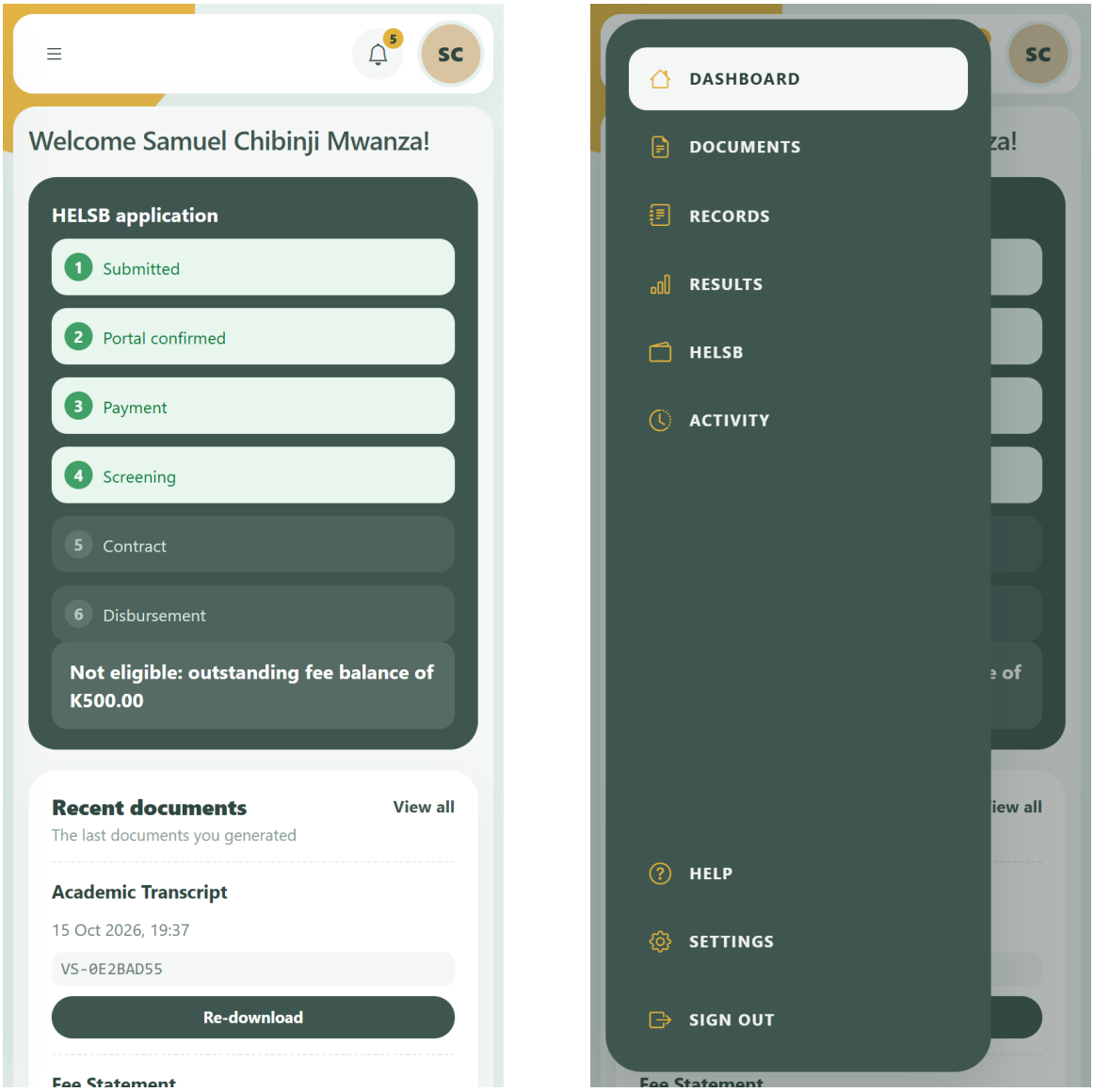


Figure 4.10: VeriScreen at phone width: the dashboard, and the menu that replaces the sidebar

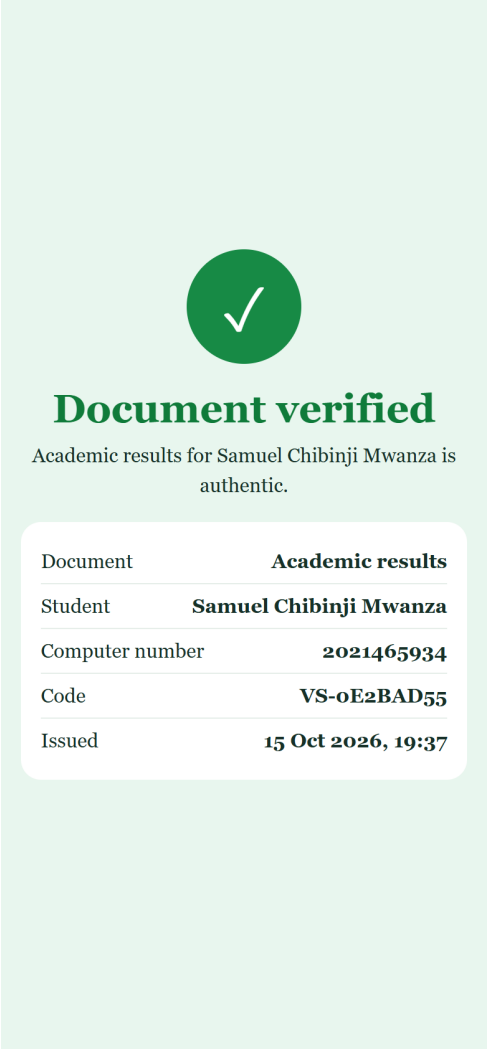


Figure 4.11: Page opened from a document QR code, including the issued time

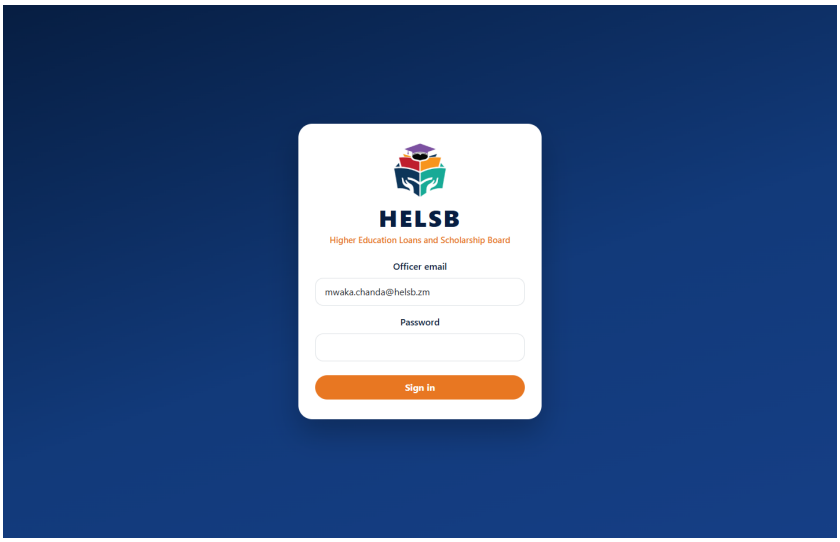


Figure 4.12: HELSB officer sign-in

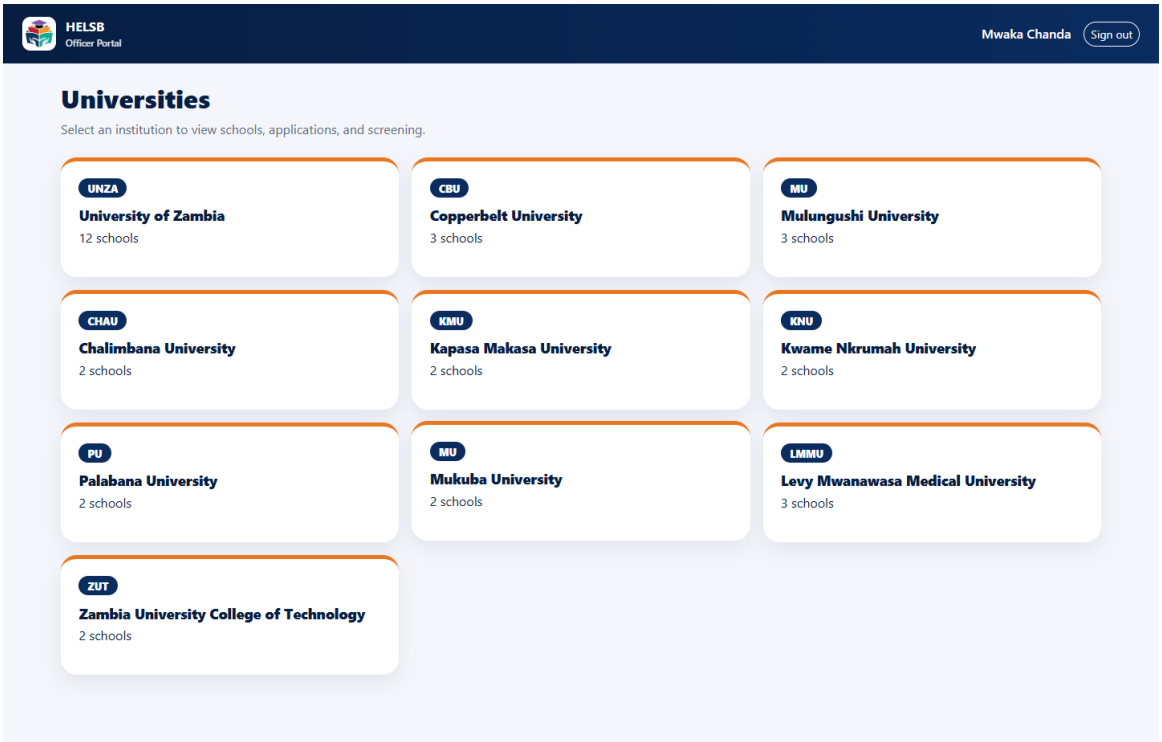


Figure 4.13: HELSB portal after sign-in: the university list

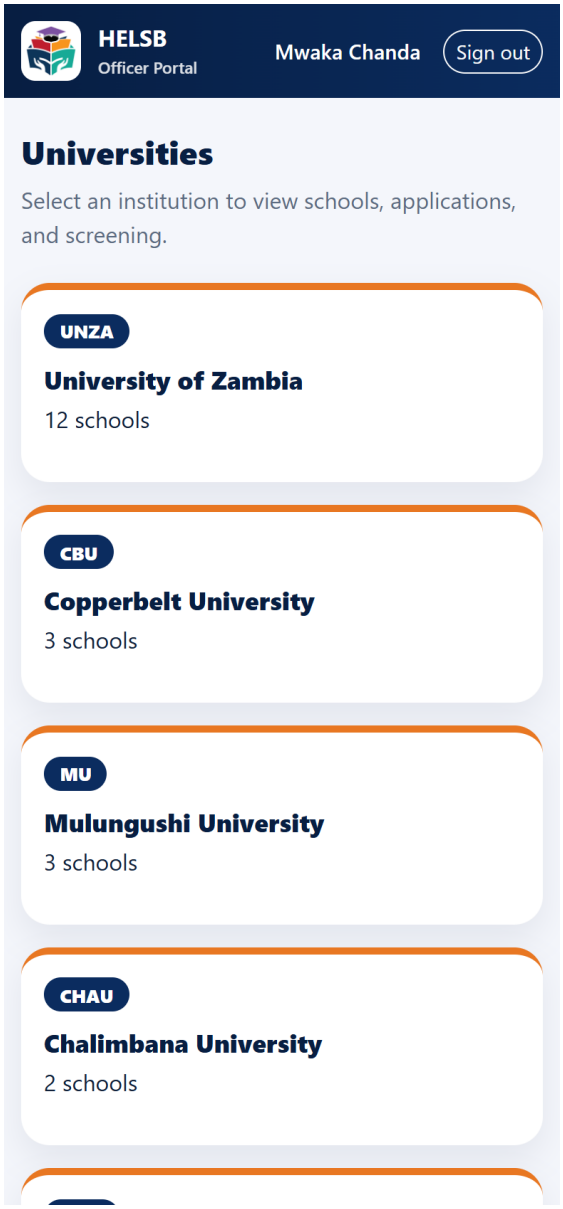


Figure 4.14: The same university list at phone width

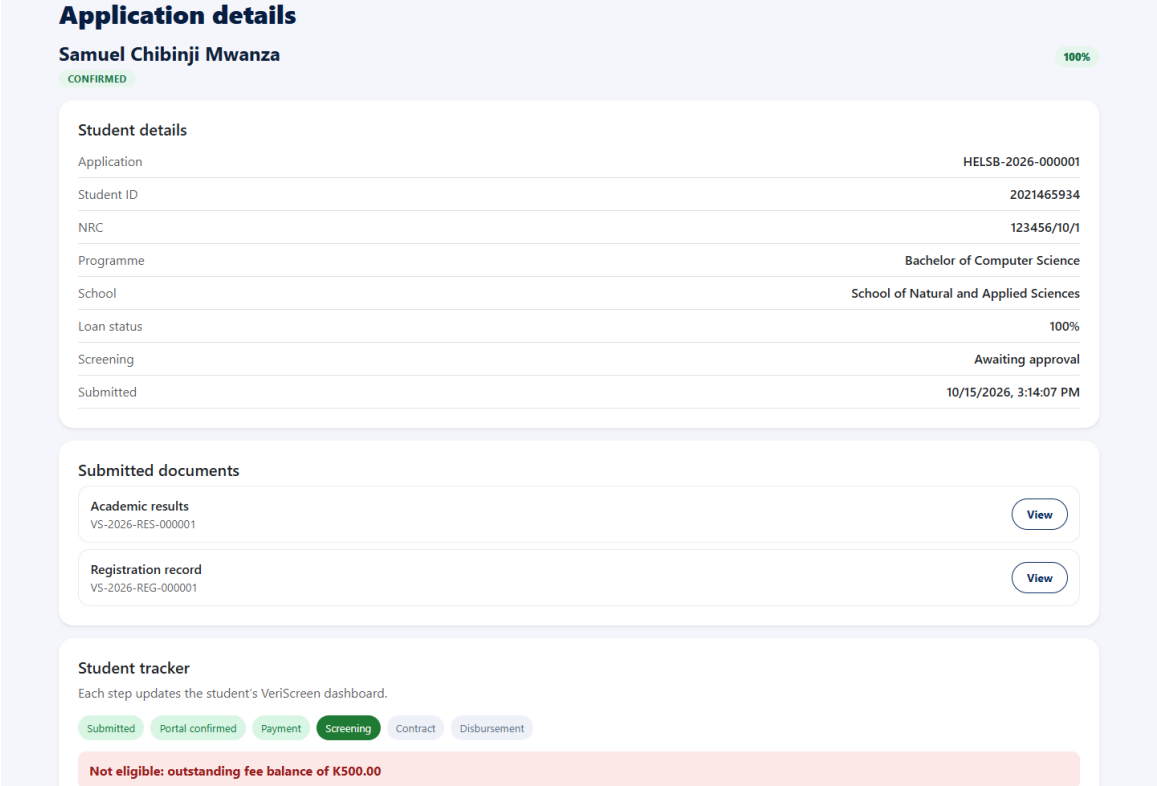


Figure 4.15: Officer view of Samuel Chibinji Mwanza at screening, with the same sentence shown on the student dashboard

4.10 Algorithms / Models / Technical Design

There is no machine-learning model. The technical core is the screening procedure in Table 4.2 and the pipeline in Table 4.3.

Table 4.3: HELSB pipeline

Stage	Officer action that enters the next stage
Submitted	Confirm in the portal
Portal confirmed	Record payment
Payment	Approve screening, which runs the explanation
Screening	Issue contract, only if the result is not “Not eligible”
Contract	Mark disbursed
Disbursement	No further action

The verification address is chosen when the document is generated. A public deployment address is used when one is configured. Otherwise the server uses a non-virtual local network address so that a phone on the same network can open the page. Virtual adapters

such as a VPN interface are skipped because a phone cannot route to them.

4.11 Chapter Summary

The design splits the student and the officer, stores each side in its own database, and defines screening as an ordered list of record checks that produce one sentence.

Chapter 5

Implementation / Development

5.1 Introduction

This chapter describes what was implemented in the completed local system.

5.2 Development Environment

The demonstration runs on a workstation with Node.js, MySQL, and two Vite development servers. The student API listens on all interfaces on port 8081. The officer API listens on port 8083. The student site is served at port 5173 and the officer site at port 5174. Each site proxies `/api` to its own API.

5.3 System Components / Modules

The student API implements sign-in, profile, registrations, fees, examinations, results, document listing, PDF download, the verification page, HELSB submission, and the internal endpoints for progress and screening. The officer API implements officer sign-in, the university and school catalogue, application retrieval, pipeline advance, and the call that requests a screening explanation. The student interface implements the dashboard tracker, documents, records, and the HELSB application view. The officer interface implements the university browser and the application tracker.

5.4 Key Implementation Details

PDF fee statements list the stored payments. The amount paid and the outstanding balance come from the fee rows. For Samuel the statement totals 14,000.00 billed, 13,500.00 paid, and a balance of 500.00. Results PDFs list the course code, grade, and a header containing the student name, computer number, and gender. The confirmation of registration is a portrait form filled from the registration and course rows. It does not invent an NRC, a birth date, or a course that is not stored.

The verification route `/verify/{documentId}` returns HTML. A known code such as Samuel's results document `VS-0E2BAD55` produces a green page titled "Document verified". An unknown code produces a red page stating that the code is not on an issued document. The JSON route `/api/verify/{documentId}` returns the same facts for a program client.

When the officer advances into screening, the officer service calls `/api/v1/internal/screen`. The student service applies Table 4.2 and returns the sentence. Both sides store it. The student dashboard polls the application and renders the sentence under the tracker.

5.5 Integration

Integration between the portals is the internal HTTP notification. The officer service sends the student computer number, the new stage, the application number, and, at screening, the explanation. The student service checks the internal key before it updates the student's application row. If the student has no row yet, one is created at the reported stage.

The university simulator is not called on every page view in the completed Node build. The demonstration record was loaded into `veriscreen_db`. Document generation and screening both read that database, so the PDF and the sentence cannot diverge from each other by reading two different copies at the same moment.

5.6 Challenges and Solutions

The first hosting path depended on Java and PostgreSQL, which ordinary shared hosting does not provide. The APIs were rewritten in Node.js with MySQL, while PostgreSQL remains available when a hosted database URL is set.

QR codes originally preferred the first network address, which on the development machine was a Cloudflare WARP adapter. A phone on Wi-Fi could not open that address. Address selection now skips virtual adapters and uses the local network address, which on the test machine was `10.31.110.236`. An already downloaded PDF keeps the old code until the student downloads the document again.

An expired officer token left the officer name on screen and showed "unable to load universities", which looked like a stopped server. The officer client now clears the session when the API returns an authentication failure and returns the officer to the sign-in page.

A narrow PDF column clipped long names. The confirmation layout was widened and set in a fixed-width face so that the stored name and school could be printed in full.

5.7 Individual Contribution (group projects)

The repository is one group product. Appendix A states the joint contribution of Samuel Chibinji Mwanza and Nosiku Mukuka. This chapter does not assign a private subsystem to either student beyond that appendix.

5.8 Chapter Summary

The implementation delivers two web portals, two APIs, two MySQL databases, PDF issue, phone verification, a six-stage tracker, and a rule-based screening sentence computed from the student record.

Chapter 6

Testing, Results and Evaluation

6.1 Introduction

Testing checked the behaviours named in the objectives. The cases below were executed against the running local system.

6.2 Testing Strategy

System tests were used. Each case had a known student or officer, a single action, and an expected sentence or page. Unit tests of every function were not the measure of success, because the objectives are end-to-end. Regression checks consisted of repeating sign-in, document verification, and screening after the server was restarted.

6.3 Test Cases and Results

Table 6.1: Representative system tests

ID	Test case	Expected result	Status
TC-01	Sign in as 2021465934	Dashboard opens for Samuel Chibinji Mwanza	Pass
TC-02	Sign in with a wrong password	Sign-in is refused	Pass
TC-03	Download Samuel's fee statement	Paid 13,500.00 and balance 500.00	Pass
TC-04	Open Samuel's results document	Name, computer number, and gender are present	Pass
TC-05	Open `/verify/VS-0E2BAD55`	Green "Document verified" page	Pass
TC-06	Open an unknown verification code	Red not-verified page	Pass
TC-07	Officer sign-in and university list	Ten universities are returned	Pass
TC-08	Advance Samuel to screening	"Not eligible: outstanding fee balance of K500.00"	Pass
TC-09	Advance Nosiku to screening	Approved sentence, fees cleared	Pass
TC-10	Advance Grace Banda to screening	Not eligible, balance K2,500.00	Pass
TC-11	Advance Peter Zulu to screening	Not eligible, balance K800.00	Pass
TC-12	Try to pass a "Not eligible" case	Pipeline stays on screening	Pass
TC-13	Continue an approved case	Contract, then disbursement	Pass
TC-14	Read the student application after TC-08	Same stage and same sentence	Pass

Nosiku's approved sentence is: "Approved: good academic standing, fees cleared, registration confirmed."

6.4 Experimental Results / Performance Results (where applicable)

No timing benchmark or load test was run. The relevant quantitative results are the record values that the sentences cite. Samuel's latest outstanding balance is K500.00. Grace Banda's is K2,500.00. Peter Zulu's is K800.00. Nosiku's latest outstanding balance is zero, her registration is confirmed, and her published grades support a GPA of at least 2.00, so the final rule applies.

6.5 User Evaluation (where applicable)

A formal user study was not conducted. Evaluation was by direct use of the student site and the officer site on the local machine, including the verification page on the Wi-Fi address of the student API.

6.6 Analysis of Results

The tests show that the document and the screening sentence are both functions of the stored record. Samuel is not described as fee-cleared, because his latest balance is K500.00. Nosiku is approved because the failing conditions do not apply. The student copy matches the officer copy after the internal notification. The failure mode that remains is environmental: a phone cannot open the verification page if it is on a different network or if the host firewall blocks port 8081.

6.7 Objective-by-Objective Evaluation

1. The student portal signs in the seeded students and shows registration, fees, examinations, and results. Objective 1 was met for the demonstration data.

2. PDF fee statements, results, and confirmation forms are generated from those rows. Objective 2 was met.

3. A known code returns "Document verified" and an unknown code does not. Objective 3 was met on the computer and on the Wi-Fi address. A physical phone still depends on network reachability and the firewall.

4. The officer portal lists institutions and advances the six-stage pipeline, and the student dashboard follows. Objective 4 was met.

5. Screening produced the K500.00, K2,500.00, K800.00, and approved sentences from the records of the four students. Objective 5 was met.

6.8 Individual Results (group projects)

The tests exercise the shared system. They are not split into personal scores. Appendix A records that the result is a group deliverable.

6.9 Chapter Summary

The fourteen system tests passed. Each screening sentence matched the fee and academic rules, and the student view showed the officer's stage and explanation.

Chapter 7

Discussion

7.1 Introduction

This chapter interprets the test results against the problem and the literature.

7.2 Interpretation of Findings

The project shows that a modest issuing service is enough to replace a screenshot for the demonstration. The verifier does not need an account. The code is either in the issued-document table or it is not. The project also shows that a loan status becomes clearer when the failing fact is written into the status. “Not eligible” alone would have hidden whether the cause was fees, registration, examinations, or grades. The K500.00 sentence can be checked against the fee statement for the same student.

7.3 Comparison with Existing Work

Compared with a portal screenshot, VeriScreen adds a server-side check and a stable document identifier. Compared with a digitally signed PDF, it is weaker against editing of the page around an intact QR code, and it is simpler to scan. Compared with a loan file that stores only a percentage, the officer portal stores the explanation that the rules produced. The system does not claim to implement a government production workflow.

7.4 Contributions of the Project

The practical contribution is a working pair of portals that a marker can sign in to and walk from a student record to a scanned page and then to an explained screening result. The technical contribution is the separation of the two databases with a narrow internal bridge, and the ordered screening rules that make the sentence deterministic.

7.5 Limitations and Implications

Because the rules are project rules, a different outstanding-balance threshold or a different GPA cut-off would change who is approved. Because the phone check only looks up the code, a forged page that still shows a genuine code would still verify. A production version would need signed document bytes, expiring codes, and a firewall and address story that does not depend on a developer's Wi-Fi. Simulated data mean that the tests do not measure accuracy against live UNZA records.

7.6 Chapter Summary

The findings support the aim inside the stated limits. Documents can be checked, and screening can be explained. The findings do not support a claim of legal equivalence to a university-signed transcript or an official HELSB decision.

Chapter 8

Conclusion and Recommendations

8.1 Introduction

This chapter closes the report by returning to the aim and the five objectives.

8.2 Summary of the Project

VeriScreen and the HELSB officer portal were designed and implemented as a group project by Samuel Chibinji Mwanza and Nosiku Mukuka. Students sign in, read a simulated university record, and download PDFs. A QR code asks the issuing server whether the document exists. Officers move an application along a six-stage pipeline. Screening writes one sentence from fees, registration, examinations, and grades, and the student sees that sentence.

8.3 Conclusions

A separate issuer can make an academic PDF checkable on a phone without giving the verifier a university password. A separate officer portal can share pipeline state with that issuer through one internal call. Explainable screening is achievable when the rules are ordered and when each rule names the record it used. The demonstration data are sufficient to show both an approval and several fee-based refusals.

8.4 Achievement of Objectives

All five objectives were achieved for the local demonstration, with the reservation that phone scanning still requires a reachable address and an open port. The aim, a working web system that issues verifiable documents and explains screening, was met.

8.5 Recommendations

The development password should be removed before any public deployment. The verification port should be allowed through the host firewall by an administrator when a phone demonstration is required. Students should download documents again after the verification address changes, because an old PDF keeps its original code. The screening threshold should be confirmed with HELSB before anyone treats the sentence as an official eligibility decision.

8.6 Future Work

Future work should verify the PDF bytes rather than only the code, expire or revoke codes, connect to a real student information system under an agreement with the university, and replace the project screening rules with the Board's published rules if those rules are made available. A short usability study with students and officers would complement the system tests in Table 6.1.

REFERENCES

1. Higher Education Loans and Scholarships Board. Official site. https://www.helsb.gov.zm.

2. ISO/IEC 18004:2015. *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*. International Organization for Standardization.

3. Jones, M., Bradley, J., and Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519.

4. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

5. Oracle Corporation. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/.

6. Meta Open Source. *React documentation*. https://react.dev.

7. OpenJS Foundation. *Express web framework for Node.js*. https://expressjs.com.

8. OWASP Foundation. *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.

Chapter A

Individual Contribution Statement (Group Projects)

Purpose of this Statement

This appendix is the group contribution statement for Samuel Chibinji Mwanza and Nosiku Mukuka. The system described in the report is a single product in one repository. The statement does not invent a private module split that was not recorded during development. Both students are responsible for the delivered student portal, officer portal, document service, and screening behaviour. A finer percentage split should be completed with the supervisor before binding if the department requires personal weights.

A.1 Project and Group Information

Item	Details
Project title	VeriScreen: A Web-Based System for Academic Document Verification and HELSB Loan Screening
Students	Samuel Chibinji Mwanza (2021465934); Nosiku Mukuka (2022030346)
Supervisor	Mrs. Monde Monica Kalumbilo Kabemba
Group size	2
Group members	Samuel Chibinji Mwanza, 2021465934; Nosiku Mukuka, 2022030346

A.2 Overall Group Project

The group built VeriScreen, a student web portal that presents a simulated university record and issues PDF documents with QR verification, and a separate HELSB officer portal that advances a loan application and explains screening in plain language. The two portals exchange pipeline state through an internal API. The demonstration includes an approved stu-

dent and students who are not eligible because of an outstanding fee balance.

A.3 Individual Role and Responsibilities

Both members worked on the shared system. The table records the areas of the product rather than a private assignment that the repository does not encode.

No.	Area of responsibility	Description
1	Student portal	Sign-in, dashboard, records, documents, and the student tracker
2	Document service	PDF layouts, QR address, and the verification page
3	Officer portal	Universities, schools, applications, and the officer tracker
4	Screening	Rules, stored explanation, and the stop on “Not eligible”

A.4 Detailed Individual Contribution

The contribution of the group, undertaken by Samuel Chibinji Mwanza and Nosiku Mukuka, covers requirements for the student and officer workflows, the literature framing in this report, the analysis and design in Chapter 4, the Node.js and React implementation, the MySQL schema and seed data, the PDF and QR integration, the pipeline notification, the screening rules, the system tests in Chapter 6, and this written report. Work that originated in an earlier Java and PostgreSQL design in the same repository was retained where it still described the domain, and was replaced where the completed demonstration required Node.js and MySQL.

A.5 Individual Deliverables

No.	Deliverable	Group contribution
1	Student web portal	Sign-in, records, documents, dashboard tracker
2	HELSB officer portal	Catalogue, application detail, pipeline actions
3	Issued PDF documents	Fee statement, results, confirmation, verification code
4	Screening explanation	Plain-language result shown to officer and student
5	This report	Account of the completed system

A.6 Contribution to the Main System or Research Output

The final output is the integrated demonstration: a student can obtain a document, a verifier can scan it, and an officer can screen the same student and produce a sentence the student can read. No component is described here as the sole work of one member.

A.7 Individual Implementation and Technical Work

The technical work includes the Express services under `veriscreen/server` and `helsb-portal/server`, the React sources under `veriscreen/frontend` and `helsb-portal/frontend`, the MySQL setup in the server database modules, PDF generation, the `/verify` page, and the screening function. Configuration for a hosted PostgreSQL database is included so that the same services can leave the local MySQL installation.

A.8 Individual Testing and Evaluation Contribution

The group executed the cases in Table 6.1, including Samuel's K500.00 result, Nosiku's approval, Grace Banda's K2,500.00 result, Peter Zulu's K800.00 result, and the verification page for `VS-0E2BAD55`.

A.9 Collaboration with Other Group Members

The student portal depends on the officer portal for stage changes, and the officer portal depends on the student API for the screening sentence. Those dependencies are inside the

group product. No external student supplied a hidden module.

A.10 Contribution Summary

1. Work completed by the group: the portals, APIs, documents, tracker, screening, tests, and this report.
2. Work completed jointly: the whole demonstration path from sign-in to disbursement.
3. Work completed by others and used here: published platform documentation cited in the references, and the department report template that structures this document. Seeded academic figures are simulated project data, not a live university extract.

A.11 Estimated Contribution

A numeric split between the two students was not logged as a separate measurement. The table therefore records the work as collaborative. The students should replace the dashes with agreed percentages before submission if the department requires them.

Project activity	**Individual**	**Collaborative**
Problem analysis	—	Group
Literature review	—	Group
Requirements analysis	—	Group
System design	—	Group
Implementation	—	Group
Testing and evaluation	—	Group
Documentation	—	Group

Note: The department template asks for approximate percentages. They are intentionally not invented here. They are not required to total 100 percent across rows once the students enter them.

A.12 Statement of Authorship and Contribution

We confirm that this statement describes the joint system truthfully. We have not knowingly assigned one member's work to the other, because the delivered system is recorded as shared work.

Student Name: Samuel Chibinji Mwanza

Computer Number: 2021465934

Signature: _____

Date: _____

Student Name: Nosiku Mukuka

Computer Number: 2022030346

Signature: _____

Date: _____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the students' participation in the group project.

Supervisor Name: Mrs. Monde Monica Kalumbilo Kabemba

Signature: _____

Date: _____

Chapter B

User Manual / Installation Guide

What must be running

Install Node.js and MySQL. Create the databases by starting each API once. The student API creates and seeds `veriscreen_db`. The officer API creates and seeds `helsb_db`. The local MySQL account used in development is `root` with the development password configured in each server environment file.

Start order

1. Start the student API in `veriscreen/server` with `node server.js`. It listens on port 8081.
2. Start the officer API in `helsb-portal/server` with `node server.js`. It listens on port 8083.
3. Start the student site in `veriscreen/frontend` with `npm run dev`. Open `http://localhost:5173/login`.
4. Start the officer site in `helsb-portal/frontend` with `npm run dev`. Open `http://localhost:5174`.

Demonstration accounts

Every development password is `password123`.

Table B.1: Accounts for the demonstration

Person	Sign-in	What screening shows
Samuel Chibinji Mwanza	2021465934	Not eligible, balance K500.00
Nosiku Mukuka	2022030346	Approved
Grace Banda	2020123456	Not eligible, balance K2,500.00
Peter Zulu	2019876543	Not eligible, balance K800.00
Mwaka Chanda	mwaka.chanda@helsb.zm	Officer
Chanda Phiri	chanda.phiri@helsb.zm	Officer

Suggested walk-through

Sign in as a student and open Records, Results, and Documents. Download a PDF again if the QR code must work on a phone, then scan it while the phone is on the same Wi-Fi. Sign in as an officer, open the University of Zambia, open the student's school, and advance the pipeline until screening. Leave the student dashboard open. It refreshes and shows the same sentence.

Troubleshooting

If the officer page says that universities could not be loaded, sign in again. An old token is rejected. If a phone cannot open the verification page, confirm that the PDF was downloaded after the server restart, that the phone is on the same Wi-Fi, and that Windows Firewall allows inbound TCP on port 8081.

Chapter C

Additional System Designs / Diagrams

No separate diagram file is embedded in this report. The architecture is fully described by Table 3.1, Table 4.2, and Table 4.3. Those tables are the design record for the completed build: two portals, two databases, one internal bridge, six pipeline stages, and six screening outcomes of which the last is approval.

Chapter D

Additional Test Cases and Results

The verification page for a known document states the document type, the student name, the computer number, and the code. Samuel's seeded documents include academic results `VS-0E2BAD55`, a fee statement `VS-1B14F7FB`, a confirmation of registration `VS-C952B77C`, and an examination slip `VS-C6B84BDF`. Opening any of those codes on the student API returns "Document verified". Samuel's results page was confirmed on `http://10.31.110.236:8081/veri` during testing. That address belongs to the development machine and will differ on another network.

Chapter E

Data Collection Instruments

No questionnaire, interview guide, or participant consent form was used. The project did not collect data from human participants. The demonstration records were authored as seed data for the system.

Chapter F

Additional Code / Configuration

The complete source is in the project repository. The behaviour described in this report is concentrated in the following places.

- `veriscreen/server/server.js` for sign-in, documents, verification, and screening.
- `veriscreen/server/pdf.js` for the PDF layouts.
- `veriscreen/server/db.js` for the student schema and seed.
- `helsb-portal/server/server.js` for the officer pipeline and the screening call.
- `helsb-portal/server/db.js` for officers and applications.
- `veriscreen/frontend` and `helsb-portal/frontend` for the two interfaces.

Environment files hold the database password and the token secret. They are not reproduced in this appendix.

Chapter G

Other Supporting Material

The department template supplied the chapter order, the declaration and approval pages, and the group-project appendix. The example references in that template, which concern crop yield and disease forecasting, were not cited because they are not sources for this system. The references in this report are the sources actually used to describe QR codes, tokens, REST, MySQL, React, Express, authentication practice, and HELSB.